VOLUME 7 OF 12

IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

MODULE 7

COLLECTIONS FRAMEWORK & GENERICS

Authored by:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

Welcome to Module 7

1. About This Module

Welcome to Volume 7 of the IT Study Hub Complete Java Programming Series, and congratulations on reaching the **Advanced Java** section! In previous modules, we used Arrays to store lists of data. However, Arrays have a fatal flaw: their size is permanently fixed. If a school uses an Array to store 500 students, and a 501st student enrolls, the program crashes.

This module introduces the **Java Collections Framework (JCF)**—a highly optimized, dynamic set of data structures that grow and shrink automatically. We will also learn **Generics**, the powerful tool that ensures our collections store exactly what we want them to, without crashing our apps at runtime.

2. Learning Outcomes

- Understand the hierarchy and core interfaces of the Java Collections Framework (List, Set, Map, Queue).
- Implement dynamic arrays using `ArrayList` and `LinkedList`.
- Eliminate duplicate data automatically using `HashSet` and `TreeSet`.
- Store and retrieve data using Key-Value pairs with `HashMap` and `TreeMap`.
- Process data in specific orders using `PriorityQueue` and `Deque`.
- Traverse collections safely using `Iterators`.
- Sort complex custom objects using `Comparable` and `Comparator` interfaces.
- Master `Generics` to build type-safe, reusable classes and methods using upper/lower bounds and wildcards.

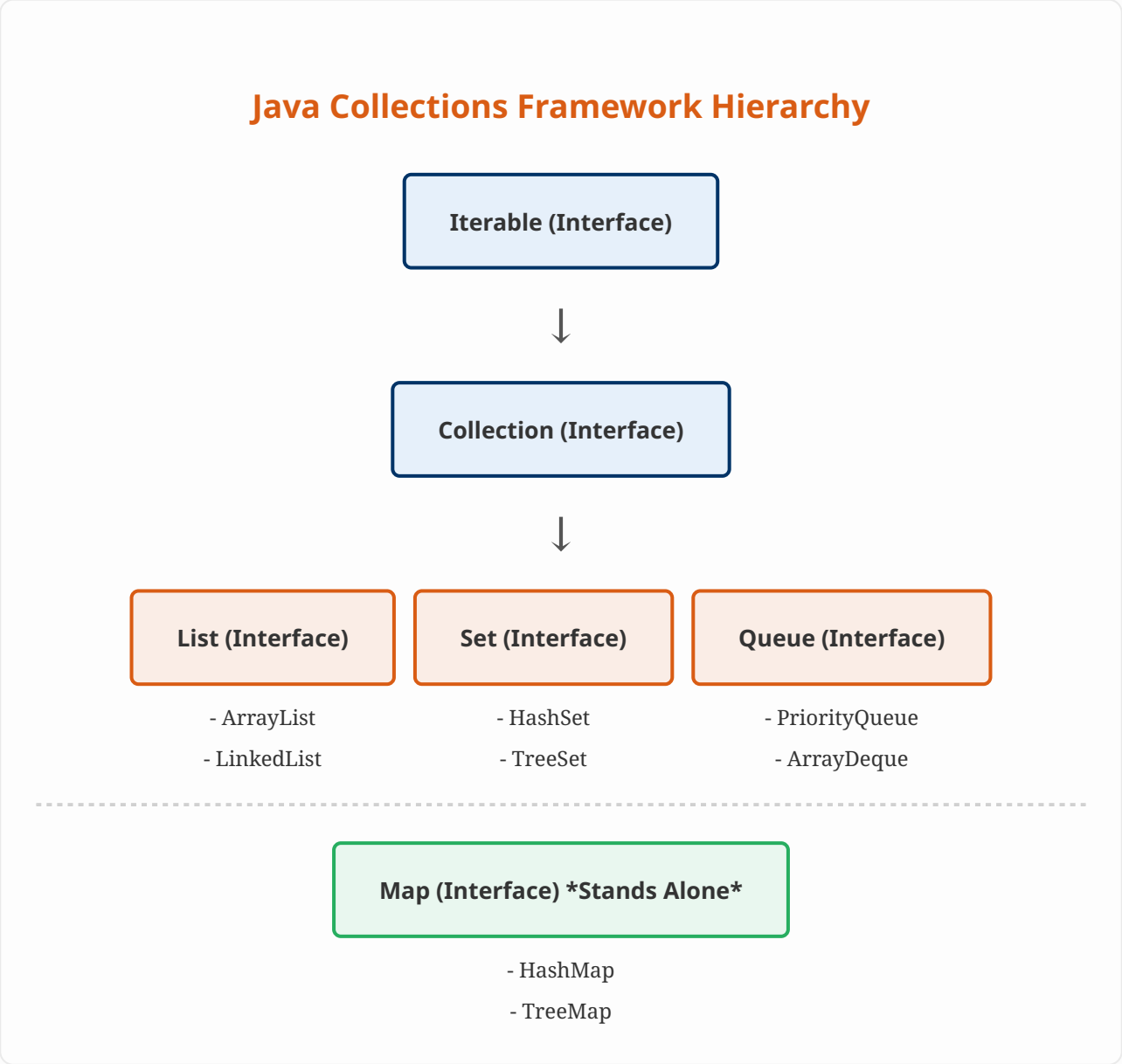
3. Module Coverage

We begin by mapping out the JCF hierarchy to understand how all data structures connect. We then deeply explore Lists, Sets, Maps, and Queues, providing comparative analysis on when to use each. Next, we conquer Iterators and Sorting mechanisms. Generics will be woven throughout and explicitly detailed at the end. Finally, you will synthesize this knowledge by building a real-world "Advanced Student Management System".

IT STUDY HUB

4. Main Notes: Introduction to Collections

A **Collection** in Java is a single object that acts as a container for holding multiple other objects (elements). The **Collections Framework** is a unified architecture representing and manipulating these collections.



List vs Set vs Map

Interface	Key Characteristic	Duplicates Allowed?	Order Preserved?
List	An ordered collection (sequence). Like an Array but dynamic.	Yes	Yes (Insertion Order)
Set	A collection that models the mathematical set abstraction.	**No**	Usually No (except Linked/Tree variants)
Map	Stores data in Key-Value pairs (e.g., ID -> Name).	Keys: No. Values: Yes.	Usually No

IT STUDY HUB

5. The List Interface (ArrayList & LinkedList)

IT STUDY HUB

Concept: ArrayList

A resizable, dynamic array.

> ***Real-Life Analogy:***
> *An Array is like a hotel with exactly 10 rooms. If an 11th guest arrives, you have to build a whole new hotel. An ArrayList is a magical hotel that automatically adds a new room whenever a new guest arrives.*

> **Simple Explanation:**
> ArrayList implements the List interface. Internally, it still uses a standard Array. However, when it gets full, Java automatically creates a new, larger array behind the scenes and copies the data over. You use `.add()`, `.get()`, and `.remove()` instead of square brackets `[]`.

Syntax / Structure:

```
List listName = new ArrayList<>();
listName.add(value);
```

Example Code:

```
import java.util.ArrayList;
import java.util.List;

public class ArrayListDemo {
    public static void main(String[] args) {
        // Create an ArrayList of Strings (Notice the  Generic type!)
        List students = new ArrayList<>();

        // Adding elements
        students.add("Alice");
        students.add("Bob");
        students.add("Charlie");
        students.add("Alice"); // Duplicates are perfectly fine in a List!

        // Accessing and Modifying
        System.out.println("Student at index 1: " + students.get(1));
        students.set(1, "Bobby"); // Update Bob to Bobby
```

```
        // Removing
        students.remove("Charlie");

        System.out.println("Final List: " + students);
        System.out.println("Total Students: " + students.size());
    }
}
```

```
Expected Output:
Student at index 1: Bob
Final List: [Alice, Bobby, Alice]
Total Students: 3
```

Code Explanation:
We use `` to enforce that this List can *only* hold Strings (this is a Generic). We use `.add()` to insert elements. Unlike Arrays which use `.length`, Collections use `.size()` to check how many items are currently inside.

Real-world Use:
Storing a list of items in an e-commerce shopping cart. The user can continuously add and remove items, so a fixed Array won't work.

Common Mistakes:

- Trying to store primitives like `int` directly: `ArrayList`. Collections can ONLY hold Objects! You must use the Wrapper class: `ArrayList`.

Best Practices:

- Always program to the Interface. Declare `List list = new ArrayList<>();` instead of `ArrayList list = new ArrayList<>();` so you can swap implementations later.

Practice Task:
Create an `ArrayList` of Integers. Add numbers 10, 20, 30. Use a loop to print them all.

Concept: LinkedList

A sequential collection where each element (node) points to the next.

> ***Real-Life Analogy:***
> *An ArrayList is a row of adjacent train cars. Finding the 5th car is instant. A LinkedList is a scavenger hunt. Clue 1 points to Clue 2. Clue 2 points to Clue 3. Finding the 5th clue takes longer because you have to follow the trail from the beginning.*

> **Simple Explanation:**
> In a LinkedList, elements are not stored next to each other in memory. Instead, each element contains the data AND a 'link' (memory address) pointing to the next element. Inserting data in the middle is extremely fast because you just change the links, you don't have to shift thousands of elements like in an ArrayList.

Syntax / Structure:

```
List list = new LinkedList<>();
```

Example Code:

```
import java.util.LinkedList;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList train = new LinkedList<>();

        train.add("Engine");
        train.add("Passenger Car");
        train.add("Caboose");

        // Specific LinkedList methods
        train.addFirst("Snow Plow");
        train.addLast("Backup Engine");

        System.out.println("Train configuration: " + train);

        train.removeFirst();
        System.out.println("After removing first: " + train);
```

```
    }
}
```

```
Expected Output:
Train configuration: [Snow Plow, Engine, Passenger Car, Caboose, Backup Engine]
After removing first: [Engine, Passenger Car, Caboose, Backup Engine]
```

Code Explanation:
We used `LinkedList` as the reference type here specifically to gain access to its special methods like `.addFirst()` and `.removeFirst()`, which are incredibly fast (O(1) time complexity).

> ***Real-world Use:***
> *Building a Music Player playlist where you frequently 'insert next song' or 'play previous song'.*

> **Common Mistakes:**
> - Using a LinkedList when you need to frequently search for items by index (e.g., `list.get(5000)`). The program will lag because it has to walk through 4,999 links to get there.

> **Best Practices:**
> - Use `ArrayList` 95% of the time. Only use `LinkedList` if your application requires constant insertions/deletions in the exact middle or beginning of massive lists.

---

6. The Set Interface (HashSet & TreeSet)

IT STUDY HUB

Concept: HashSet

An unordered collection that guarantees NO duplicates.

> ***Real-Life Analogy:***
> *A HashSet is like a bouncer at an exclusive VIP party with a guest list. If 'Alex' tries to enter twice, the bouncer simply ignores the second attempt. Furthermore, the VIPs mingle randomly inside the room; there is no 'first' or 'last' person.*

> **Simple Explanation:**
> HashSet uses a mathematical hashing algorithm to store data. Because of this, finding elements is lightning fast, but it completely destroys the order in which you inserted them.

Syntax / Structure:

```
Set set = new HashSet<>();
```

Example Code:

```
import java.util.HashSet;
import java.util.Set;

public class HashSetDemo {
    public static void main(String[] args) {
        Set emails = new HashSet<>();

        emails.add("john@test.com");
        emails.add("alice@test.com");
        emails.add("bob@test.com");

        // Attempting to add a duplicate!
        boolean isAdded = emails.add("john@test.com");

        System.out.println("Was duplicate added? " + isAdded);
        System.out.println("Email Database: " + emails);

        // Searching is extremely fast
        if(emails.contains("alice@test.com")) {
            System.out.println("Alice is registered.");
```

```
        }
    }
}
```

```
Expected Output:
Was duplicate added? false
Email Database: [bob@test.com, alice@test.com, john@test.com]
Alice is registered.
```

Code Explanation:
Notice the output order `[bob, alice, john]` does NOT match the insertion order `[john, alice, bob]`. When we tried to add John twice, the `.add()` method quietly returned `false` and kept the Set clean of duplicates.

Real-world Use:
Storing a collection of Social Security Numbers, Employee IDs, or ensuring a user can't vote twice in a poll.

Common Mistakes:

- Trying to get an item by index (e.g., `set.get(0)`). Sets do not have indices because they are unordered! You cannot fetch the 'first' element.

Best Practices:

- Use HashSet when you need blazing fast lookups (`.contains()`) and don't care what order the data prints out in.

Practice Task:
Create a HashSet of Integers. Add 5, 10, 5, 15, 10. Print the set and observe how many items it actually holds.

Concept: TreeSet

A collection that guarantees NO duplicates, AND automatically sorts the elements.

> ***Real-Life Analogy:***
> *A TreeSet is a filing cabinet where a librarian instantly alphabetizes every single folder you hand them. It keeps everything perfectly sorted, but it takes a little longer to insert new folders because the librarian has to find the right alphabetical spot.*

> **Simple Explanation:**
> TreeSet implements `SortedSet`. Every time you add an element, it gets placed in its natural sorted order (Alphabetical for Strings, Ascending for Numbers).

Syntax / Structure:

```
Set treeSet = new TreeSet<>();
```

Example Code:

```
import java.util.TreeSet;
import java.util.Set;

public class TreeSetDemo {
    public static void main(String[] args) {
        Set scores = new TreeSet<>();

        scores.add(85);
        scores.add(42);
        scores.add(99);
        scores.add(73);

        System.out.println("Auto-sorted scores: " + scores);

        // TreeSet special methods (must use TreeSet reference, not Set)
        TreeSet tree = (TreeSet) scores;
        System.out.println("Lowest score: " + tree.first());
        System.out.println("Highest score: " + tree.last());
    }
}
```

```
Expected Output:
Auto-sorted scores: [42, 73, 85, 99]
Lowest score: 42
Highest score: 99
```

Code Explanation:
Even though we inserted 85 first and 42 second, the `TreeSet` mathematically arranged them into `[42, 73, 85, 99]`. We cast the generic `Set` to `TreeSet` to unlock powerful methods like `.first()` and `.last()`.

Real-world Use:
Displaying a dynamically updating high-score leaderboard where you want the highest scores readily available without writing manual sorting algorithms.

Common Mistakes:

- Putting custom Objects (like a custom `Student` class) into a TreeSet without telling Java *how* to sort them. It will throw a `ClassCastException`. (We fix this using `Comparable` later in this module!).

Best Practices:

- Only use TreeSet if you absolutely need the data sorted at all times. It is slower than HashSet for basic adding and removing.

---

7. The Map Interface (HashMap & TreeMap)

Maps do NOT implement the `Collection` interface. They stand alone because they do not store single values; they store **Key-Value Pairs**.

HashMap Structure

Keys (Must be UNIQUE - Acts like a Set)	Values (Duplicates Allowed)
"ID_101"	"Alice Smith"
"ID_102"	"Bob Jones"
"ID_103"	"Alice Smith"

You look up the Key to instantly find the Value. Think of a dictionary: the Word is the Key, the Definition is the Value.

Concept: HashMap

Stores Key-Value pairs with fast, unordered retrieval.

> ***Real-Life Analogy:***
> *A coat check at a museum. You hand the attendant your coat (the Value). They give you a numbered ticket (the Key). When you want your coat back, you don't search the whole room. You hand them the ticket, and they instantly fetch your specific coat.*

> **Simple Explanation:**
> We use `.put(Key, Value)` to insert, and `.get(Key)` to retrieve. Keys must be strictly unique. If you `.put()` a new value using an existing Key, it permanently overwrites the old value.

Syntax / Structure:

```
Map map = new HashMap<>();
map.put(key, value);
```

Example Code:

```
import java.util.HashMap;
import java.util.Map;

public class HashMapDemo {
    public static void main(String[] args) {
        // Key is String (Username), Value is Integer (Age)
        Map userAges = new HashMap<>();

        userAges.put("Alex", 25);
        userAges.put("Sarah", 28);
        userAges.put("Mike", 32);

        // Overwriting a value using an existing key!
        userAges.put("Alex", 26);

        System.out.println("Mike's age: " + userAges.get("Mike"));
        System.out.println("Alex's updated age: " + userAges.get("Alex"));

        // Check if key exists
```

```
        if(userAges.containsKey("Sarah")) {
            System.out.println("Sarah is in the database.");
        }

        System.out.println("Full Map: " + userAges);
    }
}
```

```
Expected Output:
Mike's age: 32
Alex's updated age: 26
Sarah is in the database.
Full Map: {Alex=26, Mike=32, Sarah=28}
```

Code Explanation:
We provide TWO Generic types: ``. `Alex` was originally mapped to 25. When we executed `.put("Alex", 26)`, Java found the existing key and replaced the value.

Real-world Use:
In-memory databases. Caching user sessions: Key = SessionToken (String), Value = UserData (Object).

Common Mistakes:

- Trying to loop through a Map like a List. Maps are not simple sequences. To loop through a map, you must loop through its `keySet()` or `entrySet()`.

Best Practices:

- Use `LinkedHashMap` if you want a HashMap that remembers the order in which you inserted the items. Use `TreeMap` if you want the keys to be auto-sorted alphabetically/numerically.

Practice Task:
Create a HashMap where the Key is an Integer (Student Roll Number) and Value is a String (Grade). Add 3 entries and print the grade for Roll No 2.

8. Queue and PriorityQueue

IT STUDY HUB

Concept: Queues

Collections designed for holding elements prior to processing, usually in a First-In-First-Out (FIFO) order.

> ***Real-Life Analogy:***
> *A line at a grocery store checkout. The first person to enter the line is the first person to get served and leave the line. PriorityQueue is like an Airport VIP line: regardless of when you arrived, First Class passengers automatically skip to the front of the line.*

> **Simple Explanation:**
> Standard Queues (FIFO) use `.offer()` to add to the back, and `.poll()` to remove from the front. `PriorityQueue` mathematically sorts the elements internally so the 'highest priority' (smallest number / alphabetically first) is always at the front.

Syntax / Structure:

```
Queue queue = new PriorityQueue<>();
```

Example Code:

```
import java.util.PriorityQueue;
import java.util.Queue;

public class QueueDemo {
    public static void main(String[] args) {
        Queue pQueue = new PriorityQueue<>();

        pQueue.offer(50);
        pQueue.offer(10);
        pQueue.offer(100);
        pQueue.offer(25);

        System.out.println("Queue internal state: " + pQueue);

        // .poll() removes and returns the head of the queue
        System.out.println("Served First: " + pQueue.poll());
        System.out.println("Served Second: " + pQueue.poll());

        System.out.println("Remaining Queue: " + pQueue);
```

```
    }
}
```

```
Expected Output:
Queue internal state: [10, 25, 100, 50]
Served First: 10
Served Second: 25
Remaining Queue: [50, 100]
```

Code Explanation:
Even though we inserted 50 first, `PriorityQueue` automatically moved `10` to the front of the line because 10 is mathematically the smallest. When we `.poll()`, it dispenses 10, then 25.

> ***Real-world Use:***
> *Task scheduling in operating systems. A 'Print Queue' for a printer. A hospital triage system where critical patients are pulled first.*

> **Common Mistakes:**
>
> - Using `.add()` and `.remove()` on Queues. While they work, if the queue is capacity-restricted, they throw fatal Exceptions. Best practice is to use `.offer()` and `.poll()`, which safely return `false` or `null` if they fail.

> **Best Practices:**
>
> - Use `ArrayDeque` when you need a standard FIFO Queue (it's faster than LinkedList). Use `PriorityQueue` only when elements must be processed based on value/importance.

---

9. Iterators and Collection Traversal

IT STUDY HUB

Concept: The Iterator Interface

A universal tool used to loop through any Collection and safely remove elements during the loop.

> ***Real-Life Analogy:***
> *An Iterator is a cursor on a typewriter. It sits *between* characters. You ask it 'Is there a word next?' (hasNext). If yes, you say 'Read it and move past it' (next).*

> **Simple Explanation:**
> If you use a standard `for-loop` or `for-each` loop to remove an item from a List while you are looping through it, Java panics and throws a `ConcurrentModificationException`. An Iterator safely removes items mid-loop.

Syntax / Structure:

```
Iterator it = collection.iterator();
while(it.hasNext()) {
    Type item = it.next();
}
```

Example Code:

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IteratorDemo {
    public static void main(String[] args) {
        List names = new ArrayList<>();
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");

        Iterator iterator = names.iterator();

        while (iterator.hasNext()) {
            String currentName = iterator.next();
            System.out.println("Checking: " + currentName);
```

```
            // Safely removing an element while looping!
            if (currentName.equals("Bob")) {
                iterator.remove();
                System.out.println("-> Bob removed safely!");
            }
        }

        System.out.println("Final List: " + names);
    }
}
```

```
Expected Output:
Checking: Alice
Checking: Bob
-> Bob removed safely!
Checking: Charlie
Final List: [Alice, Charlie]
```

Code Explanation:
`iterator.hasNext()` returns true as long as there is more data. `iterator.next()` fetches the data and advances the cursor. `iterator.remove()` deletes the *last item returned by next()*. This is the only 100% safe way to delete items while looping in older Java code.

> ***Real-world Use:***
> *Filtering out expired items from a massive list of products.*

> **Common Mistakes:**
>
> - Calling `iterator.next()` twice in the same loop cycle without checking `hasNext()`. This skips elements and usually causes a `NoSuchElementException`.

> **Best Practices:**
>
> - In modern Java (Java 8+), `list.removeIf(name -> name.equals("Bob"))` is the preferred, much cleaner functional alternative to writing manual Iterator while-loops.

---

10. Sorting Custom Objects (Comparable & Comparator)

Sorting `Integers` or `Strings` is easy because Java knows 1 comes before 2, and A comes before B. But if you create a custom `Student` class and put 5 Students in a List, and call `Collections.sort(studentList)`, Java crashes. It doesn't know whether to sort by ID, Age, or Name. We must teach Java how to sort our objects.

IT STUDY HUB

Concept: Comparable (Natural Ordering)

An interface that allows an object to define its own default sorting rule.

> ***Real-Life Analogy:***
> *If you ask a classroom of students to 'sort yourselves', they know the default rule is by Last Name. That is their 'Natural Ordering'.*

> **Simple Explanation:**
> Your class must `implements Comparable`. This forces you to override the `compareTo(Object)` method. You return 1 if 'this' object is bigger, -1 if it's smaller, or 0 if they are equal.

Syntax / Structure:

```
class Student implements Comparable {
    int age;
    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.age, other.age);
    }
}
```

Example Code:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

class Player implements Comparable {
    String name;
    int score;

    public Player(String name, int score) {
        this.name = name; this.score = score;
    }

    // Define the Natural Order (Sorting by Score Ascending)
    @Override
    public int compareTo(Player other) {
```

```
        return Integer.compare(this.score, other.score);
    }

    @Override
    public String toString() { return name + ":" + score; }
}

public class ComparableDemo {
    public static void main(String[] args) {
        List players = new ArrayList<>();
        players.add(new Player("Zack", 80));
        players.add(new Player("Abby", 95));
        players.add(new Player("Mike", 50));

        System.out.println("Before: " + players);
        Collections.sort(players); // Magic happens here!
        System.out.println("After:  " + players);
    }
}
```

```
Expected Output:
Before: [Zack:80, Abby:95, Mike:50]
After:  [Mike:50, Zack:80, Abby:95]
```

Code Explanation:
Because `Player` implemented `Comparable`, `Collections.sort()` successfully read the `compareTo` logic. It compared the scores, and naturally shifted Mike(50) to the front and Abby(95) to the back.

> ***Real-world Use:***
> *Ensuring standard database entities like `Employee` always default to sorting by `employeeID`.*

> **Common Mistakes:**
> - Forgetting to add `` to the `implements Comparable` statement. Without it, you have to cast generic Objects, which is messy.

Best Practices:

- Use `Integer.compare(a, b)` or `a.compareTo(b)` for Strings inside your `compareTo` method instead of writing manual `if (a > b) return 1;` logic.

IT STUDY HUB

11. Generics: Type Safety in Java

The Problem Generics Solve

Without Generics (Pre-Java 5)

```
List myList = new ArrayList();
myList.add("Apple");
myList.add(50); // Allowed!
```

Result: ClassCastException Crash at runtime when trying to read the data!

With Generics (Modern Java)

```
List<String> myList = new
ArrayList<>();
myList.add("Apple");
// myList.add(50);
```

Result: Compiler instantly blocks the integer. 100% Type Safe!

IT STUDY

Concept: Creating Generic Classes

Writing a class that can handle any data type (T), decided entirely by the programmer when the object is instantiated.

> ***Real-Life Analogy:***
> *A Generic Class is like a blank moving box with a customizable label slot. You can buy the box and write 'Books' on the label (List), or you can buy the same box and write 'Plates' on the label (List). The box enforces whatever label you give it.*

> **Simple Explanation:**
> We use angle brackets with a placeholder letter, usually `` for Type. When designing the class, we use `T` everywhere we would normally use `int` or `String`. When we create the object in `main()`, we lock in the actual data type.

Syntax / Structure:

```
class Box {
    private T contents;
    public void add(T item) { this.contents = item; }
    public T get() { return contents; }
}
```

Example Code:

```
// Generic Class Definition
class StorageBox {
    private T item;

    public void store(T item) {
        this.item = item;
    }

    public T retrieve() {
        return item;
    }
}

public class GenericsDemo {
    public static void main(String[] args) {
```

```
        // Create a Box specifically for Strings
        StorageBox stringBox = new StorageBox<>();
        stringBox.store("Important Documents");
        System.out.println("String Box holds: " + stringBox.retrieve());

        // Create a Box specifically for Integers using the EXACT SAME CLASS
        StorageBox intBox = new StorageBox<>();
        intBox.store(999);
        // intBox.store("Text"); // COMPILER ERROR! Type Safety!
        System.out.println("Int Box holds: " + intBox.retrieve());
    }
}
```

```
Expected Output:
String Box holds: Important Documents
Int Box holds: 999
```

Code Explanation:
`StorageBox` acts as a universal template. When we do `StorageBox`, Java virtually replaces every `T` in the class with `String`. When we do `StorageBox`, it replaces every `T` with `Integer`. One class, infinite possibilities.

Real-world Use:
This is exactly how Java's `ArrayList` and `HashMap` are built under the hood. They are generic classes written by Java developers.

Common Mistakes:

- Trying to instantiate a generic type directly (e.g., `T newObj = new T();`). Java erases generic types at runtime (Type Erasure), so it doesn't know what `T` is and will crash. You can only declare variables of type `T`, not instantiate them.

Best Practices:

- Use standard generic naming conventions: `` for Type, `` for Element (used in Lists), `` for Key and Value (used in Maps).

Practice Task:
Create a generic method `printData(T data)` that accepts any data type and prints it. Call it from main with a String, an int, and a double.

IT STUDY HUB

12. Project: Advanced Student Management System

Let's synthesize everything. We will build a robust console application utilizing **ArrayList** for dynamic storage, **HashSet** to quickly check for duplicate IDs, and **Collections.sort()** with **Comparable** to sort students by Grade.

```
import java.util.*;

// 1. Student Class implementing Comparable for Natural Sorting
class Student implements Comparable {
    private String id;
    private String name;
    private double grade;

    public Student(String id, String name, double grade) {
        this.id = id;
        this.name = name;
        this.grade = grade;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public double getGrade() { return grade; }

    // Sort by Grade (Descending: Highest to Lowest)
    @Override
    public int compareTo(Student other) {
        return Double.compare(other.grade, this.grade);
    }

    @Override
    public String toString() {
        return "ID: " + id + " | Name: " + name + " | Grade: " + grade;
    }
}

public class StudentManager {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
```

```
        // 2. Collections Setup
        // List keeps the actual ordered data
        List roster = new ArrayList<>();
        // Set is used purely to track unique IDs lightning fast (O(1))
        Set existingIds = new HashSet<>();

        System.out.println("=== ADVANCED STUDENT SYSTEM ===");

        while (true) {
            System.out.println("\n[1] Add Student  [2] View Sorted Roster  [0] Exit");
            System.out.print("Choice: ");
            int choice = scan.nextInt();
            scan.nextLine();

            if (choice == 0) break;

            if (choice == 1) {
                System.out.print("Enter ID: ");
                String id = scan.nextLine();

                // 3. Prevent Duplicates using HashSet
                if (existingIds.contains(id)) {
                    System.out.println("ERROR: Student ID " + id + " already exists!");
                    continue;
                }

                System.out.print("Enter Name: ");
                String name = scan.nextLine();
                System.out.print("Enter Grade: ");
                double grade = scan.nextDouble();

                // 4. Add to Collections
                roster.add(new Student(id, name, grade));
                existingIds.add(id);
                System.out.println("Success! Added " + name);

            } else if (choice == 2) {
                if (roster.isEmpty()) {
                    System.out.println("Roster is empty.");
                    continue;
                }
                // 5. Sort using the logic we wrote in compareTo
                Collections.sort(roster);
```

```
                System.out.println("\n--- RANKED ROSTER ---");
                for (Student s : roster) {
                    System.out.println(s.toString()); // Implicitly calls toString()
                }
            }
        }
        scan.close();
    }
}
```

```
=== ADVANCED STUDENT SYSTEM ===

[1] Add Student  [2] View Sorted Roster  [0] Exit
Choice: 1
Enter ID: A101
Enter Name: John
Enter Grade: 85.5
Success! Added John

[1] Add Student  [2] View Sorted Roster  [0] Exit
Choice: 1
Enter ID: A101
ERROR: Student ID A101 already exists!

[1] Add Student  [2] View Sorted Roster  [0] Exit
Choice: 1
Enter ID: A102
Enter Name: Sarah
Enter Grade: 98.0
Success! Added Sarah

[1] Add Student  [2] View Sorted Roster  [0] Exit
Choice: 2

--- RANKED ROSTER ---
ID: A102 | Name: Sarah | Grade: 98.0
ID: A101 | Name: John | Grade: 85.5
```

Project Architecture Explanation:

1. **Why ArrayList?** We need an ordered, iterable list of students that can dynamically grow. ArrayList is the standard choice for data storage.
2. **Why HashSet?** Searching an ArrayList of 10,000 students to see if an ID exists takes O(n) time (slow). A HashSet checks if an ID exists in O(1) time (instant). We pair them together for maximum efficiency.
3. **Why Comparable?** We want the 'View Roster' command to automatically rank students. By implementing `Comparable` in the `Student` class, `Collections.sort(roster)` mathematically sorts the highest grades to the top.

IT STUDY HUB

13. Module Assessment and Interview Prep

Test your mastery of Data Structures and Generics to prepare for technical interviews.

A. Practice Questions (Test Your Logic)

1. Declare an `ArrayList` of Strings. Add three colors. Print the size of the list.
2. Convert the `ArrayList` from question 1 into a `HashSet`. What changes about how you can access the elements?
3. Write a program that adds the numbers 10, 5, 20 to a `PriorityQueue`, then polls them one by one. Write down the output order.
4. Create a `HashMap` storing `Integer` IDs as keys and `String` Names as values. Add a duplicate key with a new name and print the map.
5. Write a `for-each` loop to iterate over the keys of a `HashMap` using `map.keySet()`.
6. Create a Custom class `Employee`. Implement `Comparable` to sort them naturally by Salary.
7. Write a generic class `Pair<K, V>` that stores two variables of different generic types.
8. Write a generic method `printArray(E[] elements)` that can print an array of any object type.
9. Instantiate a `LinkedList` and use `.addFirst()` and `.addLast()`.
10. Use an `Iterator` to loop through an `ArrayList` of integers and use `it.remove()` to delete all negative numbers.
11. What happens if you try to put a primitive `double` directly into an `ArrayList<Double>`? (Hint: Think about Autoboxing).
12. Explain why `Collections.sort()` works on a `List` but not directly on a `Set`.
13. Create a `TreeSet` of Strings and add "Zebra", "Apple", "Mango". Print it.
14. Write a program using `Collections.reverse()` to reverse an ArrayList.
15. Identify the error: `List<Object> list = new ArrayList<String>();`

IT STUDY HUB

B. University Viva Questions & Answers

Q: What is the Java Collections Framework?

Ans: A unified architecture that provides interfaces and classes for storing and manipulating groups of objects dynamically.

Q: What is the difference between an Array and an ArrayList?

Ans: Arrays have a fixed size and can hold primitives or objects. ArrayLists have a dynamic size (grow automatically) and can only hold Objects (using Wrapper classes for primitives).

Q: What is the root interface of the Collection hierarchy?

Ans: The 'Iterable' interface, followed by the 'Collection' interface.

Q: Does the 'Map' interface extend the 'Collection' interface?

Ans: No. The Map interface stands alone because it stores key-value pairs, whereas standard Collections store individual elements.

Q: What is the primary difference between List and Set?

Ans: A List maintains insertion order and allows duplicate elements. A Set does not allow duplicate elements and generally does not guarantee insertion order.

Q: When should you use a LinkedList over an ArrayList?

Ans: When your application requires frequent insertions and deletions of elements in the middle or beginning of the list. ArrayList is better for frequent read/search access.

Q: How does a HashSet ensure uniqueness?

Ans: It uses a mathematical hashing algorithm (hashCode() and equals() methods) to generate a unique memory bucket for each object. If an object hashes to an already occupied bucket and equals an existing object, it is rejected.

Q: What is the difference between HashSet, LinkedHashSet, and TreeSet?

Ans: HashSet is unordered. LinkedHashSet maintains the exact order you inserted the elements. TreeSet automatically sorts the elements in their natural ascending order.

Q: What happens if you insert a duplicate key into a HashMap?

Ans: The new value completely overwrites the existing value associated with that key. The key itself remains singular.

IT STUDY HUB

Q: What is a Queue?

Ans: A collection designed for holding elements prior to processing, typically following a First-In-First-Out (FIFO) methodology.

Q: What is a PriorityQueue?

Ans: A Queue that does not follow standard FIFO. Instead, it orders elements based on their natural sorting order (or a custom Comparator), ensuring the 'smallest' or 'highest priority' element is always served first.

Q: What is the difference between Iterable and Iterator?

Ans: Iterable is an interface that allows an object to be the target of a 'for-each' loop. Iterator is an interface that provides the actual methods (hasNext(), next(), remove()) to traverse the collection.

Q: What is the difference between Comparable and Comparator?

Ans: Comparable provides a single, default 'natural' sorting sequence inside the class itself using compareTo(). Comparator provides multiple custom sorting sequences outside the class using compare(), allowing sorting by various fields (e.g., sort by Name, then sort by Age).

Q: What are Generics in Java?

Ans: A feature that allows you to specify the exact Object type a collection or class can hold using angle brackets (e.g.,), providing strict Compile-Time type safety.

Q: What is Type Erasure?

Ans: A process where the Java compiler strips out all Generic type information during compilation to maintain backward compatibility with older Java versions. The JVM at runtime has no idea what 'T' is.

Q: Can you use primitives with Generics?

Ans: No. Generics only work with Objects. You must use Wrapper classes (Integer instead of int).

Q: What does the wildcard mean in Generics?

Ans: It represents an 'Unknown Type'. It is used in method parameters when the method can operate on a Collection of any object type.

IT STUDY HUB

Q: What is the difference between and ?

Ans: is an Upper Bound wildcard; it accepts Number or any of its children (Integer, Double). is a Lower Bound wildcard; it accepts Integer or any of its parents (Number, Object).

Q: What method is used to sort a List?

Ans: Collections.sort(listName);

Q: Why is using raw types (e.g., ArrayList list = new ArrayList()) heavily discouraged?

Ans: Because it bypasses Generic type safety. The list will accept any Object, leading to catastrophic ClassCastExceptions at runtime when you try to retrieve and use the data.

C. Intermediate Interview Questions

Interview Q: Explain the internal working of an ArrayList when it gets full.

Ans: When an ArrayList reaches its capacity, it creates a new internal array that is 50% larger than the old one (capacity * 1.5). It then uses System.arraycopy() to migrate all elements to the new array, and discards the old one. This is why inserting elements can occasionally spike in processing time.

Interview Q: Explain the internal working of a HashMap (pre-Java 8).

Ans: A HashMap uses an array of 'Buckets'. When you put(Key, Value), it calculates the Key's hashCode() to determine the array index. It stores the pair in a LinkedList at that index. If multiple keys hash to the same index (a Collision), they are linked together. When get(Key) is called, it hashes the key, finds the bucket, and traverses the LinkedList using .equals() to find the exact match.

Interview Q: What major improvement was made to HashMap in Java 8?

Ans: To handle severe hash collisions, Java 8 changed the internal Bucket structure. If a single bucket's LinkedList exceeds 8 elements, the HashMap dynamically converts the LinkedList into a Balanced Red-Black Tree. This upgrades the worst-case search performance from O(n) down to O(log n).

IT STUDY HUB

Interview Q: Why must you override hashCode() if you override equals() in a custom object?

Ans: The contract between equals() and hashCode() dictates that if two objects are deemed 'equal' by .equals(), they MUST return the exact same integer hash code. If you don't override hashCode(), two logically identical Student objects will hash to different buckets in a HashSet/HashMap, causing duplicates to be erroneously allowed, breaking the Collection completely.

Interview Q: What is the difference between fail-fast and fail-safe iterators?

Ans: Fail-fast iterators (used by standard Collections like ArrayList) immediately throw a ConcurrentModificationException if the collection is structurally modified during iteration by anything other than the iterator's own .remove() method. Fail-safe iterators (used by ConcurrentHashMap) operate on a clone of the collection, allowing modifications without crashing.

Interview Q: How do you make a Collection Thread-Safe?

Ans: You can wrap it using Collections.synchronizedList(list), or use modern concurrent collections from java.util.concurrent, such as CopyOnWriteArrayList or ConcurrentHashMap, which use advanced lock-striping for higher multithreaded performance.

Interview Q: What is the difference between a Deque and a standard Queue?

Ans: A Queue strictly enforces FIFO (First-In-First-Out), inserting at the tail and removing from the head. A Deque (Double-Ended Queue) allows insertions, removals, and peeking from BOTH the head and the tail. It can function as both a Queue (FIFO) and a Stack (LIFO).

Interview Q: What does it mean that Generic wildcards are 'PECS'?

Ans: PECS stands for 'Producer Extends, Consumer Super'. It's a mnemonic for bounds. If you need to read (produce) data from a generic collection, use . If you need to insert (consume) data into a generic collection, use .

Interview Q: Can a Generic class instantiate a generic array (e.g., T[] arr = new T[10];)?

Ans: No. Because of Type Erasure, the compiler does not know what 'T' is at runtime to allocate the exact memory footprint for the array. You must use a workaround like an Object array and cast it: (T[]) new Object[10].

IT STUDY HUB

Interview Q: Why does TreeSet throw a ClassCastException if the objects don't implement Comparable?

Ans: TreeSet guarantees sorting. The moment you insert an object, it attempts to cast it to 'Comparable' to call the compareTo() method to figure out where to place it in the tree. If your object hasn't signed the Comparable contract, the cast fails and the system crashes.

IT STUDY HUB

14. Module Summary and Key Terms

Key Terms Glossary

- **Collection:** An object that groups multiple elements into a single unit.
- **Hashing:** Converting an object into an integer (hash code) to quickly determine its memory bucket.
- **Generic <T>:** A placeholder for a data type that is evaluated at compile-time to ensure strict Type Safety.
- **Type Erasure:** The compiler's process of removing generic type information to maintain backward compatibility in the JVM.
- **Wrapper Class:** An object layer around a primitive type (e.g., `Integer` for `int`), required for storing numbers in Collections.

Module Summary

In this module, you graduated from fixed-size arrays into the dynamic world of the **Java Collections Framework**. You learned that `Lists` maintain insertion order, `Sets` mathematically guarantee uniqueness, `Maps` securely bind Keys to Values, and `Queues` handle processing lines.

We explored the internal mechanics distinguishing `ArrayList` (fast reading) from `LinkedList` (fast editing), and `HashSet` (fast, random) from `TreeSet` (slower, sorted). You harnessed the power of **Generics** to forge Type Safe classes that refuse invalid data at compile-time. Finally, you learned to train Java to sort custom objects

using `Comparable`, culminating in the robust Advanced Student Management System.

WHAT COMES NEXT?

MODULE 8 — MODERN JAVA & FUNCTIONAL PROGRAMMING

Writing standard `for` loops and `If` statements to filter through Collections works, but it requires writing huge blocks of boilerplate code. In Module 8, we step into the modern era of Java 8+. You will learn to use **Lambda Expressions** and the **Stream API** to filter, sort, and process massive Collections in a single, elegant line of functional code.

IT STUDY HUB SERIES